Programação com Estrutura de Dados Avançadas

PJBL 1 - APLICAÇÃO DE DEQUE

1. ORIENTAÇÕES GERAIS

- **Equipes**. Esse trabalho deverá ser feito em grupos de três (3) ou quatro (4) componentes. Os grupos estarão previamente criados no Canvas, e cada um deverá se auto registrar em um desses grupos. Apenas serão aceitos e avaliados PjBLs enviados através dos grupos previamente cadastrados. Se, transcorridos alguns dias da divulgação do PjBL 1, ainda houver alunos não cadastrados em algum grupo, o professor irá fazer o cadastro.
- **Entrega**. Devem ser enviados os arquivos *.java com o código fonte das classes (não criar pacotes) e o arquivo *.dat com a composição ferroviária previamente criada (diferente da composição padrão, descrita mais à frente, mas construída a partir dela). Eles deverão ser enviados através da respectiva tarefa no Canvas conforme prazo lá estipulado.
- **Defesa**. A defesa/verificação do PjBL 1 será através de uma questão na Prova 1 valendo dois (2.0) pontos (a nota dessa questão não afetará a nota do PjBL1).

2. PERSISTÊNCIA DE DADOS EM JAVA

O programa a ser criado deverá fazer a persistência da composição ferroviária criada (salvá-la e recuperá-la de arquivo). Assim sendo, o código abaixo ilustra como isso pode ser realizado no Java: a classe ObjetoPersistente para salvar e recuperar um objeto (no caso, um objeto ComposicaoFerroviaria) a partir de arquivo.

CÓDIGO: PERSISTÊNCIA DE DADOS EM JAVA

```
import java.io.*; // Importa classes para entrada/saída e serialização.
// Define a classe de persistência simples.
// Se um objeto desta classe for atributo de uma classe a ser persistida,
// convém que ObjetoPersistente também seja serializável.
public class ObjetoPersistente implements Serializable {
   private String nomeArquivo; // Armazena o nome do arquivo de persistência.
   public ObjetoPersistente(String nomeArquivo) { // Construtor da classe.
      this.nomeArquivo = nomeArquivo; // Inicializa o nome do arquivo.
   }
   public String getCaminhoAbsoluto() { // Retorna o caminho completo do arquivo.
      File arq = new File(nomeArquivo); // Cria objeto File com o nome informado.
      return arq.getAbsolutePath(); // Retorna o caminho absoluto do arquivo.
   }
   public boolean existeArquivo() { // Verifica se o arquivo existe.
      File arq = new File(nomeArquivo); // Cria objeto File para o arquivo.
      return arq.exists() && arq.isFile(); // Retorna true se existe e é arquivo.
   }
   public boolean salvar(Object obj) { // Salva o objeto no arquivo.
      try (ObjectOutputStream out = // Cria fluxo de saída para objetos.
            new ObjectOutputStream( // Instancia fluxo de escrita.
               new FileOutputStream(nomeArquivo))) { // Abre arquivo para escrita.
         out.writeObject(obj); // Serializa e grava o objeto no arquivo.
         out.flush(); // Força a gravação de dados pendentes no fluxo.
         System.out.println("Dados salvos no arquivo: " + nomeArquivo + ".");
                                 // Informa sucesso na operação de salvamento.
         return true; // Retorna verdadeiro indicando sucesso.
      } catch (Exception e) { // Captura qualquer exceção ocorrida.
         System.out.println("Erro ao salvar no arquivo: " + nomeArquivo + ".");
```

```
                                // Informa falha na operação de salvamento.
            System.out.println("Motivo: " + // Exibe tipo e mensagem da exceção.
               e.getClass().getSimpleName() + " - " + e.getMessage());
            return false; // Retorna falso indicando falha.
         }
      }
      public Object carregar() { // Carrega e retorna objeto do arquivo.
         if (!existeArquivo()) { // Verifica se o arquivo não existe.
            System.out.println("Arquivo inexistente: " + nomeArquivo + ".");
                                // Informa que o arquivo não foi encontrado.
            System.out.println("Caminho esperado: " + getCaminhoAbsoluto());
                                // Mostra onde o arquivo deveria estar.
            return null; // Retorna null indicando ausência de dados.
         }
         try (ObjectInputStream in = // Cria fluxo de entrada para objetos.
               new ObjectInputStream( // Instancia fluxo de leitura.
                  new FileInputStream(nomeArquivo))) { // Abre arquivo para leitura.
            Object obj = in.readObject(); // Desserializa o objeto do arquivo.
            System.out.println("Dados carregados do arquivo: " + nomeArquivo + ".");
                                // Informa sucesso na operação de leitura.
            return obj; // Retorna o objeto carregado.
         } catch (Exception e) { // Captura qualquer exceção ocorrida.
            System.out.println("Erro ao carregar o arquivo: " + nomeArquivo + ".");
                                // Informa falha na operação de leitura.
            System.out.println("Motivo: " + // Exibe tipo e mensagem da exceção.
               e.getClass().getSimpleName() + " - " + e.getMessage());
            return null; // Retorna null indicando falha.
         }
      }
}
```

Para utilizar a classe ObjetoPersistente na recuperação e salvamento do arquivo com a composição ferroviária, a classe ComposicaoFerroviária deve ter os métodos indicados abaixo, que utilizam métodos da classe ObjetoPersistente.

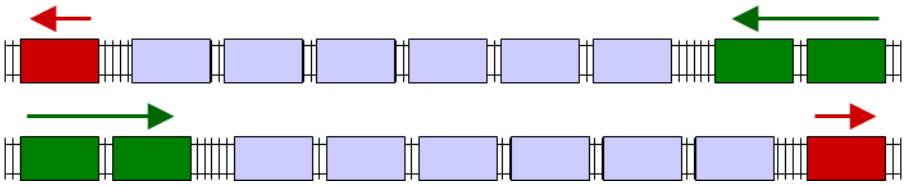
```
import java.io.*;

public class ComposicaoFerroviaria extends Deque implements Serializable {
   ObjetoPersistente ArqComp;

   public ComposicaoFerroviaria(int N, String nomeArquivo) {
      super(N);  // Chama o construtor de Deque.
      ArqComp = new ObjetoPersistente(nomeArquivo);
      carregar(); // Carrega arquivo da composição ferroviária se existir.
   }

   private void salvar() { // Salva a composição ferroviária em arquivo.
      ArqComp.salvar(this);
   }

   private void carregar() {
      ComposicaoFerroviaria cf = (ComposicaoFerroviaria) ArqComp.carregar();
      if (!(cf==null)) { // Se o objeto foi recuperado, atualiza composição ferroviária.
      // É necessário redefinir os atributos do "deque pai" com os dados da
      // composição carregada.
         this.front = cf.front;
         this.rear = cf.rear;
         this.ptr = cf.ptr;
         this.size = cf.size;
         this.N = cf.N;
         this.data = cf.data;
      }
```

```
:    // Restante do código.
```

3. ENUNCIADO

3.1. Descrição

Esse trabalho visa desenvolver uma classe que permita criar composições ferroviárias e gerenciar algumas de suas características (tamanho, comprimento, peso total etc.). Como estrutura, uma composição pode ser entendida como sendo um deque: pode ser montada adicionando-se ou retirando-se vagões de ambas as extremidades da composição.



Os três tipos de vagões a serem considerados na montagem da composição são os abaixo.

- Locomotiva.
  Comprimento entre 18 m 23 m; 100 a 200 toneladas de peso (quanto mais potência, maior o peso); potência entre 2000 HP e 6000 HP.
- Passageiro (Vagão de passageiros).
  Comprimento de 22 m a 26 m; 30 a 50 toneladas de peso total (vagão mais passageiros); até 50 passageiros.
- Carga (Vagão de carga).
  Comprimento de 12 m a 19 m; de 15 a 30 toneladas de peso total (vagão mais carga); considerar 75% do peso total do vagão como sendo carga para fins de cálculo.

Notar que apenas as características sublinhadas acima são específicas: todos os três tipos têm comprimento e peso. Isso sugere usar herança (ver diagrama de classes mais à frente).

Será considerado que a quantidade mínima de HPs por tonelada de peso (HPT) da composição para se obter um desempenho mínimo/aceitável é HPT = 1.05 HP/Ton.

3.2. Requisitos

Feita a descrição acima, os requisitos específicos para a classe "Composição Ferroviária" (derivada da classe Deque) e o respectivo programa de teste são apresentados a seguir.

1. Métodos para carregamento de composição pré-existente a partir de arquivo e para gravação da composição em arquivo. Utilizar métodos da classe Persistente.

2. Método para criar automaticamente, via código, uma composição padrão. Esse método deverá popular a composição exatamente com os vagões especificados no Anexo. Essa composição será utilizada como base para avaliar os resultados dos métodos. Toda vez que este método for chamado, a composição anterior deve ser substituída por esta e salva em arquivo.

3. Métodos para inserção e remoção de vagões (subclasses locomotiva, carga e passageiro) por qualquer extremidade, com atualização automática da composição no arquivo após cada inserção/remoção. Para inserção e remoção, utilizar os métodos disponíveis na classe Deque no respectivo método da classe Composição Ferroviária.

4. Método para recuperação do número de vagões da composição e da quantidade de cada tipo de vagão.

5. Método para contabilização do comprimento total da composição, considerando que o espaço entre vagões é de 2 m, e o peso total da composição.

6. Método para contabilização do número de passageiros e do total de carga transportada.

7. Método para verificação se a potência da(s) locomotiva(s) adicionada(s) à composição (sempre adicionar locomotivas iguais) é suficiente para o desempenho mínimo desta; essa verificação deve mostrar a relação potência/peso da composição e indicar se a potência da(s) locomotiva(s) é suficiente, ou então quanta potência ainda falta, sugerindo quantas locomotivas iguais às já incluídas na composição devem ser adicionadas.

8. Os métodos dos itens 4 a 7 devem ser utilizados por um outro método específico (Diagnóstico da composição) para mostrar o diagnóstico da composição com os resultados desses métodos (4 a 7), os quais não serão instanciados diretamente no programa principal.

9. Métodos para retornar os dados do primeiro e do último vagão.

a. Classe "pai" Vagão com um método imprime() para atributos comuns.

b. Classes "filhas" Locomotiva, Passageiro e Carga com método imprime() (override) utilizando super.imprime(): impressão dos dados comuns mais os específicos.

10. A classe Composição Ferroviária deve ser obrigatoriamente uma classe derivada da classe Deque criada na prática PEDA_06_LL_Deque_Pratica.pdf (além da classe ObjetoPersistente fornecida).

a. Adicionar apenas métodos específicos à nova classe, os quais deve fazer uso dos métodos das classes base (pais).

b. Não alterar os métodos da classe deque original (não é necessário), mas apenas utilizá-los!

Os requisitos para o programa de teste seguem abaixo.

11. Implementação do programa conforme o diagrama de classes mostrado a seguir (*item 3.3*).

12. Criar um programa de teste com as opções abaixo que que crie um objeto "Composição Ferroviária " de um dado tamanho e o gerencie utilizando um menu com as seguintes opções:

a. criar composição padrão (item 3 acima);

b. inserir vagão (na frente, ao final e o tipo com os respectivos atributos);

c. remover vagão (do início ou do final);

d. apresentar a descrição da composição (requisito 8);

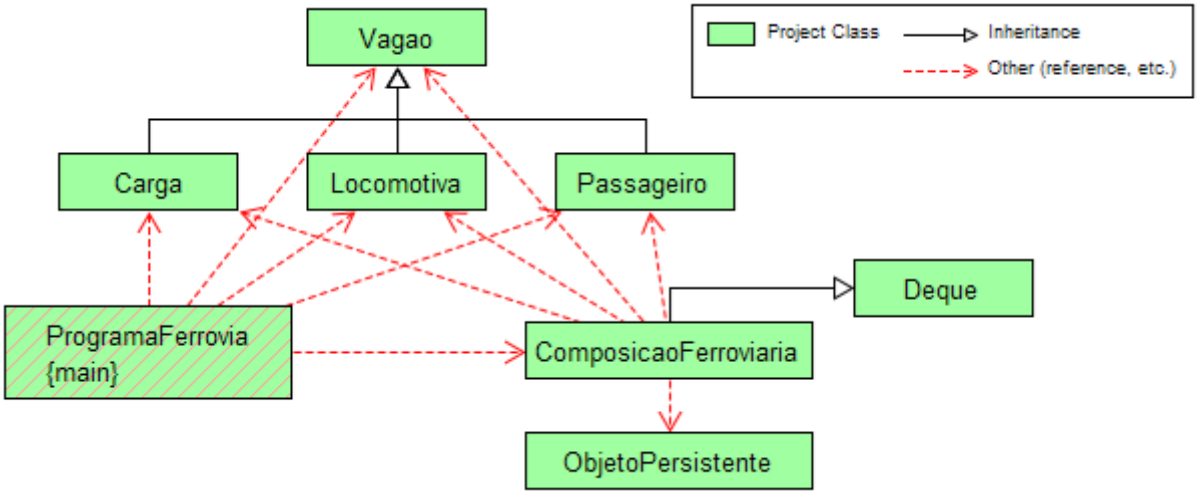
e. apresentar os dados do primeiro vagão;

f. apresentar os dados do último vagão;

g. terminar.

3.3. Digrama de Classe

O diagrama de classes simplificado (gerado pelo jGrasp) correspondente aos requisitos do problema é mostrado abaixo. As linhas cheias em preto indicam herança a partir da classe com o pequeno triângulo. As tracejadas indicam uso/referência.



3.4. Algumas Orientações

3.4.1. Para a Adição de Vagões

À composição ferroviária serão acrescentados objetos todos derivados da classe Vagão ("abstrata"): Locomotiva, Passageiro e Carga. É só criar os objetos descendentes de vagão e utilizar os métodos já existentes do deque para adicioná-los (ou removê-los) da composição: o polimorfismo gerencia isso.

3.4.2. Para a Contabilização de Valores

Algumas contabilizações necessitam saber o tipo de vagão ao se percorrer a composição. Abaixo, um exemplo da contabilização.

```
public double contarCargaTotal() {
   double total = 0;
   rewind();
   for (int i = 0; i < getSize(); i++) {
      Object obj = next();
      if (obj instanceof Carga)
         total += ((Carga) obj).carga;
   }
   return total;
}
```

3.4.3. Para a Sobrescrita de Métodos

Exemplo.

```
import java.io.Serializable;
public class Vagao implements Serializable {
   protected double comprimento; // Comprimento do vagão em metros.
   protected double peso;        // Peso do vagão em toneladas.
     :
   public void imprime() {
      System.out.println("Comprimento: " + comprimento +
                         " m, Peso: " + peso + " toneladas.");
   }
}

# Subclasse Locomotiva
public class Locomotiva extends Vagao {
   private double potencia; // Potência da locomotiva em HP.
     :
   @Override
   public void imprime() {
      System.out.println("Tipo: Locomotiva.");
      System.out.println("Potência: " + potencia + " HP.");
      super.imprime();
   }
}
```

3.4.4. Para criar a Composição Padrão

Para criar a composição padrão, a anterior deve ser apagada.

```
   public void criarComposicaoPadrao() {
      while (!isEmpty())
         deleteLast();
      for (int i = 0; i < 30; i++) {
         addLast(new Carga(17, 20));
      }
         :
   }
```

3.4.5. Para Recuperar a Composição

A classe ObjetoPersistente salva o objeto inteiro. Ao se recuperar a composição, a composição na memória deve ter seus atributos redefinidos conforme objeto carregado, conforme o código já mostrado no item 2, e repetido abaixo.

```
   private void carregar() {
      ComposicaoFerroviaria cf = (ComposicaoFerroviaria) ArqComp.carregar();
      if (!(cf==null)) { // Se o objeto foi recuperado, atualiza composição ferroviária.
      // É necessário redefinir os atributos do "deque pai" com os dados da
      // composição carregada.
         this.front = cf.front;
         this.rear = cf.rear;
         this.ptr = cf.ptr;
         this.size = cf.size;
         this.N = cf.N;
         this.data = cf.data;
      }
```

4. RUBRICA

A correção do programa entregue levará em conta os valores abaixo para cada requisito atendido.

#	Requisito	Valor
1	Método para carregamento/gravação.	0.5
2	Método para criar composição padrão.	2.0
3	Métodos para inserção e remoção de vagões.	0.5
4	Métodos para recuperação do número de vagões e seus subtipos	0.5
5	Método para contabilização do comprimento e peso totais.	0.5
6	Método para contabilização do número de passageiros e carga total.	0.5
7	Método para verificação se a potência é suficiente e recomendações	0.5
8	Métodos para diagnóstico (#4 a #7).	2.0
9	Método para retornar os dados de um vagão	0.5
10	Implementação do programa com as classes Deque e Persitente indicadas.	1.5
11	Diagrama de classes implementado corretamente.	0.5
12	Programa de teste correto	0.5
	TOTAL	**10.0**

Também haverá descontos conforme abaixo.

#	Desconto	Valor
1	Carregamento inicial incorreto/ausente.	-3.0
2	Uso de outra classe Deque distinta da solicitada.	-2.0
3	Código não original: plágio, IA etc.	-10.0
4	Uso de bibliotecas de ED do Python.	Até -4.0
5	Outro recurso não visto em sala.	Até -4.0

5. REFERÊNCIAS

As referências abaixo podem ser utilizadas para complementar as informações do material de aula.

5.1. Biblioteca Virtual

- PUGA, Sandra Gavioli; RISSETTI, Gerson. **Lógica de programação e estruturas de dados, com aplicações em Java**. 3º ed. São Paulo, SP: Pearson, 2016. *E-book*. Disponível em: https://plataforma.bvirtual.com.br/Leitor/Publicacao/41600/pdf/0. Acesso em: 21 fev. 2025.

5.2. Minha Biblioteca

- GOODRICH, Michael T.; TAMASSIA, Roberto. **Estruturas de dados e algoritmos em Java**. Porto Alegre: Grupo A, 2013. *E-book*. ISBN 9788582600191. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788582600191/. Acesso em: 21 fev. 2025.
- SZWARCFITER, Jayme L.; MARKENZON, Lilian. **Estruturas de Dados e Seus Algoritmos**. Rio de Janeiro: LTC, 2010. *E-book*. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/978-85-216-2995-5/. Acesso em: 11 set. 2022.

ANEXO - COMPOSIÇÃO PADRÃO

Abaixo, os vagões a serem incluídos na composição inicial. Como os tipos têm mesmos atributos, é possível utilizar um processo repetitivo para inclusão dos vagões de cada tipo na composição. Essa composição será utilizada para validar os cálculos do programa.

(75% peso)

Vagão	Qtde	Comp. (m)	Peso (Ton)	HP	Passag.	Carga (ton)
Locomotiva	1	20	150	2500	-	-
Passageiros	50	24	40	-	30	-
Carga	30	17	20	-	-	15